



UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MÉXICO

CENTRO UNIVERSITARIO UAEM VALLE DE MÉXICO

**Desarrollo de un Modelo de Aprendizaje Incremental de
Redes Neuronales Artificiales Aplicado al Conjunto de
Datos Optdigit**

PROTOCOLO DE INVESTIGACIÓN

P r e s e n t a

C. González Hernández Luis Ángel

**Asesor: Dr. Víctor Manuel Landassuri Moreno
Co-Asesora: Ing. Carmen Lucía Bustillo Hernández**

Atizapán de Zaragoza, Edo. de Méx. Enero 2025



Índice

Índice de figuras	2
1. Introducción	4
2. Planeamiento del Problema	5
3. Objetivos	6
3.1. Objetivo General	6
3.2. Objetivos Particulares	6
4. Hipótesis	6
5. Justificación	6
6. Delimitación	7
7. Consecuencias	7
8. Marco Teórico	7
8.1. Optical Digit	7
8.2. Redes Neuronales Artificiales	8
8.2.1. Función de Activación	9
8.3. Redes Neuronales de Perceptrón Multicapa	11
8.4. Algoritmo de Retropropagación	12
8.5. Redes Neuronales Convolucionales	12
8.6. Aprendizaje Incremental	12
8.6.1. Algoritmos de Aprendizaje Incremental	13
8.7. Estado del Arte	14
8.7.1. Introducción	14
8.7.2. Aprendizaje Incremental en la Robótica	14
8.7.3. Aprendizaje Incremental en Redes Neuronales Convolucionales	14
8.7.4. Aprendizaje Incremental en el Tráfico Cifrado	14
9. Metodología	15
10. Organización del Capitulado	15
11. Diagrama de Gantt	16
A. Fases de la Retropropagación	16
A.1. Propagación hacia adelante	16
A.2. Propagación hacia atrás	16
Referencias	17

Índice de figuras

1. Conjunto de Datos Optical Digit	7
2. Red Neuronal Artificial Básica	8
3. Puertas Lógicas	8
4. Puerta Lógica XOR	9
5. Función Escalonada	9
6. Función Sigmoide	10
7. Función ReLU	10
8. Función Softmax	11
9. Función Tangente Hiperbólica	11
10. Red Neuronal Multicapa	12
11. Dos enfoques tradicionales del aprendizaje incremental.	13

12. Diagrama de Gantt	16
---------------------------------	----

Desarrollo de un Modelo de Aprendizaje Incremental de Redes Neuronales Artificiales Aplicado al Conjunto de Datos Optdigit

16 de mayo de 2025

Resumen

Una red neuronal artificial es un modelo de Aprendizaje Automático inspirado en el funcionamiento de las redes neuronales biológicas. Está compuesta por neuronas artificiales (o nodos) organizadas en capas (entrada, oculta y salida), donde la información se procesa mediante transformaciones matemáticas no lineales.

El aprendizaje de estos modelos se realiza a través de un proceso de entrenamiento, en el cual los datos atraviesan la red capa por capa. En cada neurona se calcula una función de activación, se estima la pérdida comparando la salida con el valor real, y luego se aplica el algoritmo de retropropagación para ajustar los pesos y sesgos con el objetivo de minimizar dicha pérdida. Este proceso se repite hasta que la pérdida converge.

En el enfoque tradicional, el modelo se entrena con un conjunto inicial de datos y, ante la llegada de nueva información, debe reentrenarse desde cero utilizando todos los datos disponibles. Este proceso puede resultar ineficiente y costoso computacionalmente, especialmente con grandes volúmenes de datos.

El aprendizaje incremental, una rama de la Inteligencia Artificial, permite actualizar el conocimiento del modelo sin necesidad de reentrenarlo completamente con los datos históricos [2]. En este trabajo se emplearán Redes Neuronales Artificiales enfocadas en la clasificación de dígitos manuscritos, utilizando el algoritmo de backpropagation en redes Multicapa Perceptron. Se explorará una estrategia de duplicación de pesos para simular mecanismos de memoria a corto y largo plazo, con el objetivo de mejorar los resultados presentados en [2].

Palabras clave: Aprendizaje incremental, Redes Neuronales Artificiales, Clasificación de dígitos.

1. Introducción

La inteligencia artificial (IA) es un campo del conocimiento que busca desarrollar máquinas capaces de emular el comportamiento y razonamiento humano, permitiendo una interacción casi indistinguible de la que se tendría con otro ser humano. Esta área no solo se enfoca en la creación de sistemas inteligentes, sino también en desarrollar herramientas que faciliten y optimicen las actividades cotidianas.

Un subcampo de la IA es el Aprendizaje Automático (Machine Learning), que estudia algoritmos capaces de aprender tareas de manera autónoma. Dentro de este campo, las Redes Neuronales Artificiales (RNAs) son una de las técnicas más reconocidas, pues utilizan procesos matemáticos para resolver tareas complejas. Las RNAs han demostrado ser útiles en áreas como la predicción de capacidades de redes 5G basadas en el tráfico diario [19], así como en la clasificación de materiales como metales y rocas mediante lógica difusa [17].

Las RNAs están conformadas por neuronas artificiales que simulan las biológicas. En este contexto, los procesos químicos que ocurren en el cerebro se imitan computacionalmente a través de señales que viajan entre las neuronas artificiales, que en adelante llamaremos "neuronas". Esta estructura distribuida y paralela otorga a las RNAs una notable capacidad de aprendizaje [11].

En el contexto del aprendizaje automático, cuando una técnica como las RNAs se enfoca en aprender una tarea específica, se considera un *algoritmo lineal sin memoria*. Desde sus inicios, este ha sido uno de los enfoques más utilizados en las RNAs [6]. Sin embargo, cuando se necesita incorporar nueva información sin reentrenar todo el modelo, surge el concepto de Aprendizaje Incremental, que busca incluir nuevos datos sin tener que entrenar el modelo desde cero.

Un aspecto derivado del aprendizaje incremental es el concepto de memoria en la IA, que explora cómo un algoritmo de aprendizaje automático puede olvidar información previamente adquirida al incorporar nuevos datos. Esta problemática, análoga a la pérdida de memoria humana, plantea desafíos tanto para máquinas como para seres humanos. En respuesta, se han diseñado diversos enfoques para mitigar este fenómeno. Un ejemplo es el trabajo de [2], que propone el uso de RNAs con pesos dobles, donde la primera capa de pesos se comporta como memoria a corto plazo y la segunda como memoria a largo plazo. Los experimentos realizados

en [2] muestran una mejora en tareas de aprendizaje incremental, reduciendo la pérdida de información en comparación con implementaciones previas como el algoritmo Learn++ [9, 4].

Este trabajo de investigación tiene como objetivo explorar nuevas configuraciones de pesos duplicados, ampliando y extendiendo los resultados obtenidos en [2].

2. Planeamiento del Problema

Las Redes Neuronales Artificiales (RNAs), en especial las *Multilayer Perceptrons* (MLP), tienen la capacidad de aprender tareas y resolver problemas de predicción y clasificación. Utilizando algoritmos de aprendizaje como el *Backpropagation*, es posible ajustar los pesos de una RNA para que pueda abordar una tarea específica. Las MLP son una forma común de RNA, compuestas por múltiples capas de neuronas, donde la capa de entrada recibe los datos y la capa de salida proporciona las predicciones. Las capas intermedias, también conocidas como capas ocultas, realizan transformaciones no lineales que permiten a la red aprender representaciones complejas de los datos. Sin embargo, muchas de las tareas que se resuelven en la vida cotidiana generan información adicional con el tiempo. Un ejemplo de esto es el comportamiento de una serie financiera o la predicción del clima en una determinada región. En este contexto, surge el aprendizaje incremental, un enfoque poco explorado que permite que el modelo aprenda nueva información sin la necesidad de reentrenar todo el modelo con los datos antiguos y nuevos.

En los modelos actuales de aprendizaje automático, cuando se utiliza un conjunto de datos para entrenar un modelo específico, dicho modelo es funcional solo para ese conjunto y la información que representa. Sin embargo, al ser necesario incorporar nueva información al modelo, se debe recolectar esa nueva información, añadirla a la ya existente y luego reentrenar todo el modelo para integrar los datos nuevos.

En este contexto, si una red entrenada con un primer conjunto de datos d_1 debe entrenarse con un segundo conjunto de datos d_2 , al hacerlo, se perderá el conocimiento adquirido por d_1 . Si no se utiliza un modelo de aprendizaje incremental, se debe combinar d_1 y d_2 en un solo conjunto y reentrenar la RNA para incorporar el nuevo conocimiento (d_2). En cambio, si se emplea el aprendizaje incremental, la RNA se entrena inicialmente con d_1 y luego con d_2 , manteniendo una mínima pérdida de información de d_1 . Si en el futuro se obtiene más información del problema (d_3), se entrenará la RNA con d_3 usando el modelo incremental, minimizando la pérdida de datos de d_1 y d_2 .

Este trabajo se basa en la investigación de [2], que utiliza una configuración de pesos duplicados en la RNA. En esta configuración, los pesos de la RNA se duplican y se asignan diferentes tasas de aprendizaje a las capas. La primera capa, asociada a una tasa de aprendizaje alta, simula la memoria a corto plazo al aprender rápidamente una nueva tarEste trabajo se basa en la investigación de [2], que propone una estrategia para mejorar la capacidad de aprendizaje incremental de las Redes Neuronales Artificiales (RNA) mediante la duplicación de pesos. En este enfoque, cada conexión de la red neuronal cuenta con dos versiones de sus pesos: una que simula la memoria a corto plazo y otra que representa la memoria a largo plazo.

La memoria a corto plazo utiliza una tasa de aprendizaje alta, lo que le permite adaptarse rápidamente a nueva información, pero también la hace más propensa a olvidar conocimientos previos. Por otro lado, la memoria a largo plazo se entrena con una tasa de aprendizaje baja, aprendiendo de manera más lenta pero reteniendo mejor la información adquirida anteriormente.

Durante el entrenamiento, ambos conjuntos de pesos se actualizan en paralelo según sus respectivas tasas de aprendizaje. Luego, al momento de realizar una predicción, los resultados se obtienen a partir de una combinación de ambos conjuntos de pesos. Esta estrategia permite que la red neuronal integre tanto el conocimiento reciente como el conocimiento acumulado, emulando el funcionamiento complementario de los sistemas de memoria biológica. ea y olvidar más rápido la información anterior. La segunda capa, con una tasa de aprendizaje baja, simula la memoria a largo plazo, aprendiendo más lentamente y olvidando menos la información previa. Al integrar ambas capas, la RNA pondera la salida para considerar tanto la nueva información adquirida como la olvidada en ambas capas de pesos.

El problema principal del aprendizaje incremental en [2] es que, a medida que se incorporan nuevos conjuntos de datos, se pierde progresivamente el conocimiento adquirido de los primeros conjuntos, lo cual se vuelve menos útil si en el futuro se tienen 10 o 20 etapas de entrenamiento incremental con nuevos datos.

Por lo tanto, es crucial explorar nuevas configuraciones de RNAs que mejoren los métodos actuales para reducir la cantidad de información olvidada a medida que llega nueva información. Al igual que en investigaciones anteriores, este trabajo se basará en los conceptos de memoria a corto y largo plazo. Sin embargo, en lugar de duplicar los pesos y tener dos tasas de aprendizaje, se propondrá la idea de crear más copias de los pesos, cada una con una tasa de aprendizaje distinta.

3. Objetivos

3.1. Objetivo General

Diseñar una red neuronal artificial para aprendizaje incremental basada en el principio de la memoria a corto y largo plazo, buscando usar más de dos capas de pesos duplicados para el reconocimiento de dígitos, y con una menor pérdida de información de trabajos previos.

3.2. Objetivos Particulares

1. Implementar el algoritmo mostrado en [2] para el reconocimiento de dígitos con aprendizaje a corto y largo plazo con los parámetros que ahí se indican.
2. Obtener el conjunto de datos de Optical Digits, limpiar los datos y prepararlos según lo indicado con [2].
3. Separar el conjunto de entrenamiento y de prueba de acuerdo a lo que se explica en el artículo [2] y probar el primer código implementado en miras de comprobar los resultados previamente mostrados en [2].
4. Tomar como base el algoritmo implementado, y extenderlo para permitir mas de dos pesos duplicados, aplicando el conjunto de datos previamente mostrado.
5. Comparar ambas implementaciones en busca de una reducción significativa de las tasas de aprendizaje con respecto a trabajos previos en la literatura.

4. Hipótesis

La inclusión de más de dos capas de pesos duplicados, cada una con sus respectivas tasas de aprendizaje, puede reducir el olvido de la información previamente aprendida en un modelo de aprendizaje incremental utilizando redes neuronales artificiales del tipo MLP, cuando se aplica el algoritmo de Backpropagation.

5. Justificación

Las redes neuronales artificiales permiten el aprendizaje automático y la resolución de distintos problemas. Sin embargo, como se mencionó anteriormente, las técnicas de aprendizaje automático presentan una deficiencia importante: cuando se incorporan nuevos bloques de datos, se observa un deterioro en el rendimiento del aprendizaje y un olvido de la información previamente aprendida [2].

Aunque los resultados obtenidos hasta ahora no han sido perfectos, la memoria a corto plazo de las redes neuronales tiende a olvidar con el tiempo, lo cual es una limitación. En contraste, los seres humanos pueden aprender nuevas tareas o información de un problema sin olvidar de manera significativa lo que previamente aprendieron. Aunque biológicamente los humanos tienen una estructura cerebral que les permite aprender y retener mejor la información, actualmente no existe ningún procedimiento que permita modificar la estructura del cerebro para mejorar la retención y reducir el olvido.

Desde un punto de vista computacional, no hay limitaciones inherentes que impidan experimentar con nuevas configuraciones de redes neuronales artificiales (RNAs) para lograr que toda la información ingresada en el modelo se acumule, sin problemas de almacenamiento, y sin olvidar la información previa. Esto podría ser beneficioso en diversas aplicaciones.

En un escenario en el que no se utilice aprendizaje incremental, la llegada de nueva información implicaría la necesidad de volver a entrenar todo el modelo con la información anterior y la nueva. Esto sería un proceso costoso en términos de cómputo y energía, ya que el entrenamiento de una RNA es uno de los cuellos de botella principales, lo que significa un alto consumo de recursos. En cambio, si solo se entrena con la nueva información, se podrían reducir tanto el tiempo como el consumo energético.

Existen varias herramientas que permiten codificar redes neuronales artificiales utilizando librerías preexistentes. En esta investigación, se destacan dos plataformas principales: Microsoft Azure y Google Colab. Azure ofrece una máquina virtual para programar en Python, mientras que Google Colab proporciona un entorno similar, pero de forma gratuita. Ambas herramientas permiten trabajar con Python, un lenguaje de programación libre y fácil de depurar, ideal para el desarrollo de aplicaciones de Machine Learning.

Una de las librerías más populares para Machine Learning es TensorFlow, que facilita la creación y entrenamiento de redes neuronales, permitiendo detectar patrones y razonamientos. Debido a sus capacidades y a su

compatibilidad con Deep Learning, TensorFlow será la librería utilizada en esta investigación, particularmente porque es compatible con Keras, un framework de alto nivel que se ejecuta sobre TensorFlow. Keras simplifica los procesos de experimentación rápida y es ideal para su ejecución en plataformas como Google Colab.

6. Delimitación

En la presente investigación, se utilizarán exclusivamente redes neuronales del tipo perceptrón multicapa (MLP). Cabe señalar que este no es el único tipo de red neuronal existente, ya que también existen Redes Neuronales Convolucionales (RNC), Redes Neuronales Recurrentes (RNR) y Redes de Base Radial (RBR) [16]. Además, no se abordarán técnicas de optimización como los algoritmos genéticos, limitándose únicamente a explorar la mejora en el rendimiento de redes neuronales con más de dos capas duplicadas de pesos en un contexto de aprendizaje incremental. Asimismo, el estudio se restringe a las redes mencionadas, sin incluir modelos de redes neuronales profundas ni el uso de otros conjuntos de datos.

7. Consecuencias

Si el experimento tiene éxito, se espera que ocurran las siguientes consecuencias:

1. Se reducirá el olvido de la información previamente aprendida.
2. El tiempo de aprendizaje será menor.

Como se mencionó previamente, será posible incorporar nuevos datos sin necesidad de reentrenar el modelo con toda la información existente.

Las tareas de clasificación o predicción podrán integrar información nueva en las redes neuronales artificiales sin la necesidad de volver a entrenar el modelo con todos los datos históricos, lo que permitirá reducir el cuello de botella asociado al proceso de entrenamiento.

8. Marco Teórico

8.1. Optical Digit

Es un conjunto de datos utilizado en el área de reconocimiento de patrones, el motivo de su creación fue para la investigación de reconocimiento de caracteres ópticos de caracteres (OCR). Su representación es de imágenes en la escala de grises con un tamaño de 8x8 píxeles (como se muestra en la Figura 1), cada imagen representa un número en el rango de 0 a 9, dicho dataset contiene un total de 5620 imágenes, apesar de esto el dataset tiene un balance de clases, lo que quiere decir que cada clase (en este caso cada dígito) tiene la misma cantidad de muestras.

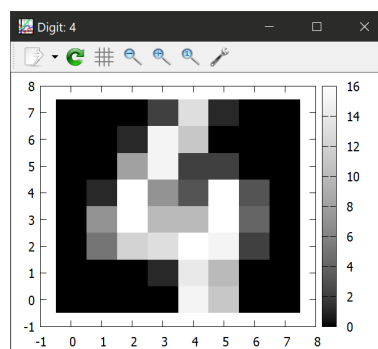


Figura 1: Conjunto de Datos Optical Digit

Este conjunto de datos se utiliza en diferentes áreas de la inteligencia artificial, tales como:

1. Clasificación de dígitos [13].
2. Reconocimiento de patrones [7].

3. Comparación de modelos [12].

4. Aprendizaje supervisado [8].

Al usar este tipo de conjunto de datos se obtienen algunas ventajas que por su simplicidad y tamaño permite que los test sean más rápidas que con otros.

8.2. Redes Neuronales Artificiales

Las redes neuronales artificiales (RNA) son modelos computacionales dentro de la Inteligencia Artificial que contienen unidades de procesamiento simples llamadas neuronas. Estas se inspiran en el cerebro humano, basándose en la conectividad entre neuronas y el aprendizaje que pueden tener. Un perceptrón o neurona (artificial) solo resuelve problemas lineales y tiene la siguiente forma:

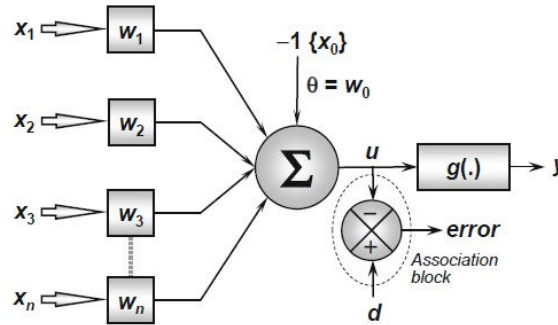


Figura 2: Red Neuronal Artificial Básica

Donde Σ es la representación matemática de la neurona. $x_1, x_2, \dots, x_n$ son las variables de entrada a la red, y $w_1, w_2, \dots, w_n$ son los pesos con los cuales se ponderan las entradas, es decir, se multiplican cuando la información entra en la neurona. Posteriormente, se suman todos esos valores: $w_1x_1 + w_2x_2 + w_3x_3$.

Al revisar la fórmula anterior, se puede observar que se parece a la operación de una regresión, la cual es: $y = w_0 + w_i x_i$. De esta manera, internamente la neurona realiza una regresión lineal. El parámetro que permite a la neurona trazar una recta cruzando el eje y en el plano cartesiano (eje de las ordenadas) es conocido como sesgo (del inglés *bias*). Este valor se agrega a la conexión y usualmente se le asigna un valor de 1.

Agregando este nuevo valor a la fórmula, queda de la siguiente manera: $y = \Sigma w_i x_i + w_0 b$, donde b es el sesgo.

Es importante tener en cuenta que una sola neurona solo puede resolver problemas lineales, como las puertas lógicas AND u OR. Sin embargo, no es capaz de manejar problemas no lineales, como la puerta XOR, ya que no puede separar correctamente los datos. Para resolver este tipo de casos más complejos, es necesario usar redes neuronales de múltiples capas, conocidas como Perceptrón Multicapa, que permiten una mejor clasificación combinando varias capas neuronales.



Figura 3: Puertas Lógicas

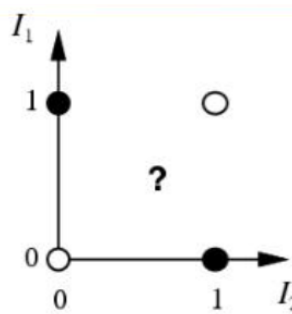


Figura 4: Puerta Lógica XOR

Además del uso de 2 o más capas dentro de la neurona, es indispensable el uso de una función de activación (Sección 8.2.1) que permite pasar la información de una neurona a otra dentro de un rango específico, lo cual se describirá en la siguiente sección.

8.2.1. Función de Activación

Se utilizan cuando el modelo de RNA contiene dos o más neuronas; además, proporciona al modelo una salida no lineal. Este tipo de funciones es especialmente útil en problemáticas donde las relaciones entre los datos de entrada no son lineales, e.g. Si se tiene un modelo con tres entradas que representan distintas características (como temperatura, presión y humedad), es probable que la relación entre estas variables y la salida esperada no sea una simple combinación lineal. En estos casos, una red neuronal sin funciones de activación no podría captar esa complejidad, limitando severamente su capacidad de aprendizaje.

Las funciones de activación, como ReLU, sigmoide o tangente hiperbólica, introducen no linealidad en la red, permitiéndole identificar patrones más complejos. Sin estas funciones, aunque la red tenga varias capas, todas operarían como una única transformación lineal, incapaz de resolver problemas donde la relación entre los datos no sea directa ni proporcional.

Además, estas funciones ayudan en el modelado probabilístico, ya que sus salidas pueden estar acotadas dentro de un rango, como de 0 a 1 en el caso de la función sigmoide, lo cual es útil para representar probabilidades [14].

Al hablar de funciones de activación, se deben comentar las más comunes:

■ Función Escalonada:

Esta función se utiliza para problemas de clasificación, ya que su salida es binaria, es decir, 0 o 1.

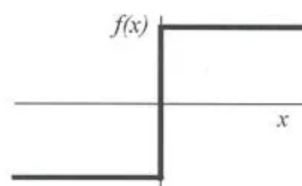


Figura 5: Función Escalonada

Dicha función está representada por:

$$f(x) = \begin{cases} 0 & : x < 0 \\ 1 & : x \geq 0 \end{cases}$$

■ Función Sigmoide:

Esta función es una de las más comunes, ya que su salida es un rango de 0 a 1, lo que permite interpretarla como una probabilidad. e.g. Si la salida es 0.8, se interpreta como un 80 % de probabilidad que algo ocurra.

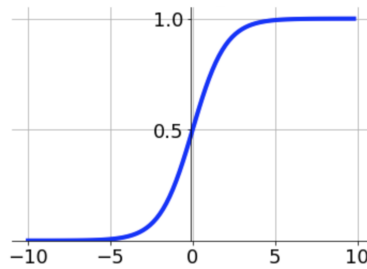


Figura 6: Función Sigmoide

Está representada por la siguiente fórmula:

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

Sin mencionar que este tipo de funciones es ajustable, lo cual es una característica importante del algoritmo de retropropagación (Sección: 8.4).

■ **Función de Unidad Rectificada Lineal (ReLU):**

Esta función es la más utilizada en RNAs ya que supera los problemas de desvanecimiento del gradiente, además de ser más rápida en el entrenamiento.

Es una función lineal que, cuando es positiva, toma el valor de la entrada, y cuando es negativa, toma el valor de 0.

- *Desvanecimiento del gradiente: Es un problema que ocurre cuando los gradientes (ajustes que aprende la red) se vuelven extremadamente pequeños durante el entrenamiento, casi cero, lo que provoca que la red no pueda aprender o las capas iniciales de la red no se actualizan a los nuevos datos.*

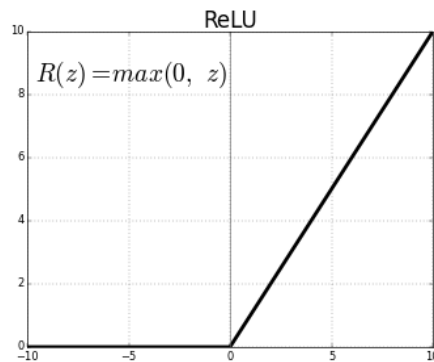


Figura 7: Función ReLU

Está representada por la siguiente fórmula:

$$f(x) = \begin{cases} 0 & : x < 0 \\ x & : x \geq 0 \end{cases}$$

■ **Función Softmax:**

Esta función se utiliza en la capa de salida de la red neuronal, ya que transforma las salidas en una representación probabilística, de tal manera que la suma de todas las probabilidades sea 1.

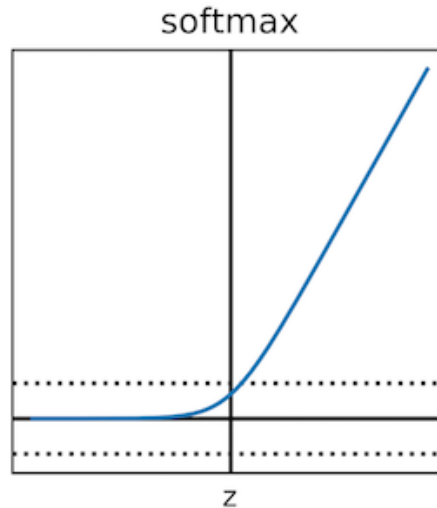


Figura 8: Función Softmax

Su representación matemática es:

$$f(z)_j = \frac{e^{z_j}}{\sum_{K=1}^K e^{z_k}}$$

■ **Función Tangente Hiperbólica:**

Esta función es similar a la función sigmoide, pero su rango es de -1 a 1. También sufre de problemas de desvanecimiento del gradiente, lo que puede dificultar el entrenamiento de redes neuronales profundas.

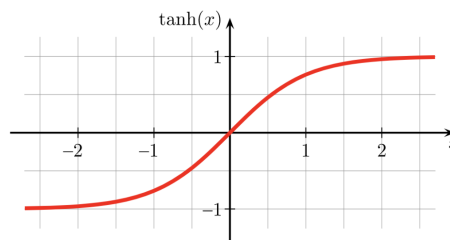


Figura 9: Función Tangente Hiperbólica

Su representación matemática es:

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Las redes neuronales presentan diversas utilidades que ayudan a resolver problemas como no linealidad, mapeo entrada-salida, aprendizaje robusto a errores en los datos de entrenamiento, entre otros. Existen varios tipos de redes neuronales, como las redes neuronales de perceptrón multicapa y redes neuronales convolucionales, que se describirán brevemente más adelante.

8.3. Redes Neuronales de Perceptrón Multicapa

Las Redes Neuronales de Perceptrón Multicapa (MLP) constan de tres o más capas: una capa de entrada, una o más capas ocultas y una capa de salida. En las capas ocultas, se pueden tener más de una fila de neuronas, que son las encargadas de realizar las operaciones para eliminar la linealidad de los datos. Como se comentó anteriormente en la sección 8.2.1, la linealidad de los datos se elimina con las funciones de activación, que modifican los parámetros de la red, permitiendo la elaboración de un plano tridimensional con el cual se puede encontrar la solución al problema planteado.

Además, como se explicó anteriormente, no es recomendable trabajar con una sola neurona debido a los problemas al resolver tareas como XOR, donde se requieren dos líneas rectas para clasificar el problema correctamente.

Como se puede observar en la Figura 10, para este caso se cuenta con una MLP que consta de 4 capas: una de entrada, dos ocultas y una de salida. En las capas ocultas y de salida se lleva a cabo el procesamiento de las funciones de activación, mientras que en la capa de entrada no se aplica ninguna función de transferencia, ya que simplemente representa las entradas al modelo.

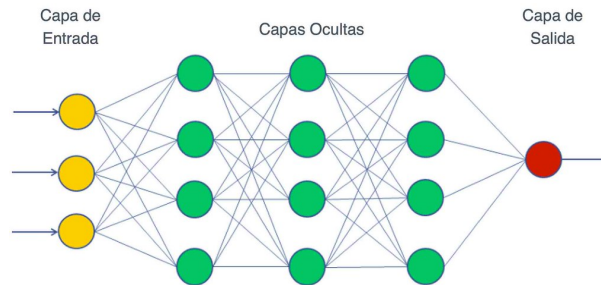


Figura 10: Red Neuronal Multicapa

8.4. Algoritmo de Retropropagación

El algoritmo de retropropagación es un algoritmo de aprendizaje que permite que una red neuronal autoajuste todos sus parámetros para aprender una representación interna de la información que está procesando. Llegó para resolver la limitante del perceptrón, que solo resuelve problemas lineales, y se extiende a redes más complejas, es decir, a problemas no lineales.

Mediante este algoritmo se obtienen las derivadas parciales del gradiente y los pesos, los cuales se utilizan para optimizar la red neuronal. También se deben calcular las derivadas del sesgo, que indican en qué capa se encuentra el error. El uso de estas derivadas parciales permite encontrar el error, y lo que realiza el algoritmo es retroceder hasta la neurona donde se encuentra el error, regresando desde la capa de salida hasta la capa de entrada.

Para poder realizar todo este proceso de retropropagación, es necesario contar con una función de activación diferenciable.

Como se ha visto existen distintos tipos de funciones de activación y el algoritmo de retropropagación es aplicable a todas ellas. Una función de activación diferenciable hace que la función computada por una red neuronal sea diferenciable, ya que la red en sí misma computa solamente composiciones de funciones. La función de error también se vuelve diferenciable. [15].

El algoritmo de retropropagación calcula gradientes mediante un proceso en dos fases:

1. **Feed-forward:** Propagación de entradas hacia adelante 2. **Backpropagation:** Propagación de errores hacia atrás

Para entender mejor lo siguiente véase el Apéndice A (Secciones A.1 a A.2).

8.5. Redes Neuronales Convolucionales

Las Redes Neuronales Convolucionales (CNN, por sus siglas en inglés) son redes profundas con una estructura especial. Están conformadas por tres tipos de capas: convolucionales, de agrupación y completamente conectadas.

En las capas convolucionales, el filtro detecta características específicas dentro de la imagen de entrada, como bordes, colores, o texturas, y genera un mapa de características. En las capas de agrupación, se reduce la resolución espacial de la imagen, lo que permite reducir la cantidad de parámetros y el costo computacional. Finalmente, las capas completamente conectadas están encargadas de realizar la clasificación.

Las CNN son muy útiles para el reconocimiento de imágenes, y se han utilizado en una amplia variedad de aplicaciones, desde la visión por computadora hasta la medicina y el reconocimiento de voz.

8.6. Aprendizaje Incremental

Con el paso del tiempo, la tecnología ha evolucionado, lo que ha llevado a un aumento en la cantidad de datos disponibles. El aprendizaje automático ha experimentado avances significativos, y los datos se generan y procesan con mayor frecuencia.

Se puede definir una tarea de aprendizaje como incremental si los ejemplos de entrenamiento utilizados para resolverla se presentan de manera secuencial, generalmente uno a la vez. Si los resultados no son urgentes, este tipo de tareas puede ser resuelto mediante algoritmos de aprendizaje no incremental [6]. Un área donde el aprendizaje incremental es especialmente útil es en la *robótica*, ya que estos sistemas requieren entrenamiento constante [6].

Este enfoque de aprendizaje fue inspirado por la forma en que los seres humanos aprenden y es más rápido, lo que llevó a su adopción en el campo del aprendizaje automático.

Con el tiempo, el aprendizaje incremental se ha convertido en un paradigma del aprendizaje automático, donde el sistema toma nuevos ejemplos y los agrega a los ya aprendidos. A medida que el sistema aprende, los ejemplos previos pueden ser reemplazados por los nuevos [11].

8.6.1. Algoritmos de Aprendizaje Incremental

Un algoritmo de aprendizaje incremental se define por los siguientes criterios:

1. Ser capaz de aprender y actualizarse con cada nuevo dato, etiquetado o no etiquetado.
2. Conservar el conocimiento adquirido previamente.
3. No requerir acceso a los datos originales.
4. Ser capaz de generar nuevas clases o clusters cuando sea necesario, así como dividir o fusionar clusters según lo requiera el entorno.
5. Tener una naturaleza dinámica, adaptándose a un entorno cambiante [1].

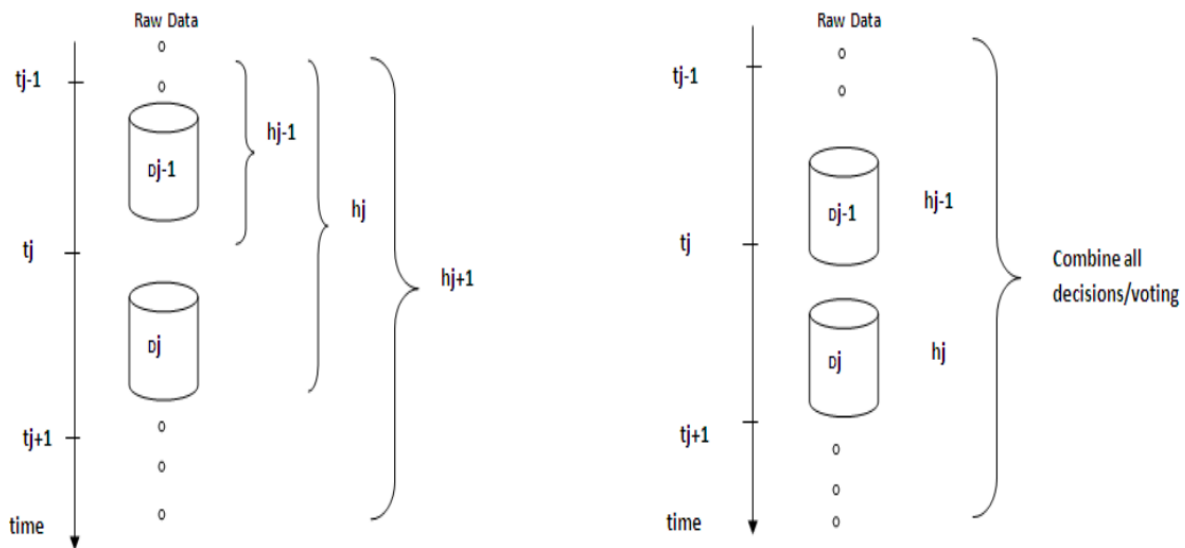


Figura 11: Dos enfoques tradicionales del aprendizaje incremental.

Como se observa en la Figura 11, el primer enfoque consiste en la acumulación de datos, donde, al recibir una nueva porción de datos D_j , se descarta la hipótesis h_{j-1} y se genera una nueva hipótesis h_j basada en todos los datos disponibles hasta ese momento. En el segundo enfoque, al recibir una nueva porción de datos D_j , se desarrolla una única hipótesis nueva o un conjunto de hipótesis nuevas basadas en los nuevos datos. Finalmente, se puede usar un mecanismo de votación para combinar todas las decisiones de las diferentes hipótesis y obtener la predicción final.

El aprendizaje incremental tiene la ventaja de no requerir almacenamiento de los datos previos, ya que el conocimiento se guarda en las hipótesis generadas durante el proceso de aprendizaje.

“Un algoritmo de aprendizaje es incremental si, para cualquier muestra de entrenamiento dada:

$$e_1, e_2, \dots, e_s$$

produce una secuencia de hipótesis

$$h_0, h_1, \dots, h_n$$

tal que h_{i+1} depende solo de h_i y de la muestra actual $e[6]$.”

Como se observa, estos algoritmos permiten que la inteligencia artificial realice tareas de predicción de manera más eficiente.

Un ejemplo de esta metodología es el proyecto *COBWEB*, que categoriza el número de clusters y la pertenencia de dichos clusters utilizando una métrica probabilística global. Este proceso implica agregar nuevas categorías, actualizando las probabilidades con los nuevos datos recolectados [5].

8.7. Estado del Arte

8.7.1. Introducción

En la actualidad, el aprendizaje incremental es una de las áreas de Inteligencia Artificial más investigadas, enfocada en el desarrollo de modelos que se mantengan en constante aprendizaje. A lo largo de los años, se han realizado diversas investigaciones en este campo, destacando aplicaciones en áreas como la salud, la robótica, entre otras.

El enfoque en específico en esta área es la creación de modelos que puedan aprender nuevos conocimientos y puedan implementar el *Lifelong learning* (aprendizaje a lo largo de la vida o aprendizaje de por vida), el cual como se ha visto ha beneficiado al aprendizaje humano, también se prevé que en el área de la inteligencia artificial beneficie a los modelos de aprendizaje automático y poder obtener una mejor precisión en los resultados obtenidos.

Como se verá a continuación, se presentarán algunas investigaciones que han implementado el aprendizaje incremental y han creado sus propios modelos, los cuales han favorecido el área de aplicación en la que se encuentran.

8.7.2. Aprendizaje Incremental en la Robótica

En [10] se tiene el objetivo de mejorar su prototipo de Wearable-Walker-Exoskeleton, el cual consta de una prótesis la cual trabaja por medio de redes neuronales artificiales, las cuales entran en un constante aprendizaje conforme el usuario va utilizando dicha prótesis. El motivo de implementar aprendizaje incremental a dicho proyecto es por el constante conocimiento que van adquiriendo las redes neuronales, al ser un proyecto de robótica, debe mantenerse en constante aprendizaje para ofrecer una mejor experiencia al usuario.

8.7.3. Aprendizaje Incremental en Redes Neuronales Convolucionales

Xing Wei menciona en su investigación [18] que en el área de redes neuronales convolucionales (RNA convolucionales) se tiene la problemática de que, al recibir un nuevo conjunto de aprendizaje, el modelo entrenado tiende a olvidar los datos previos. En varios de sus objetivos a largo plazo, se plantea que, por medio del aprendizaje incremental, un coche autónomo pueda circular por diversas vías urbanas. Sin embargo, para lograrlo, el modelo entrenado debe mantener el conocimiento previamente adquirido y, al mismo tiempo, seguir obteniendo nuevos conocimientos.

Para ello, en dicha investigación se proponen distintas soluciones, entre las cuales destacan:

1. El diseño de una red de aprendizaje incremental.
2. Un entrenamiento de aprendizaje incremental de extremo a extremo para su adaptación.
3. Una alineación multinivel, la cual permitiría reducir la discrepancia y así alinear los datos previos con los nuevos.

Como resultado de la investigación, se obtuvo un nuevo tipo de entrenamiento con aprendizaje incremental, el cual propone dividir el conjunto de datos (dominio) en distintos dominios. Al combinar esto con el aprendizaje incremental, se obtiene un resultado más coherente y eficiente.

8.7.4. Aprendizaje Incremental en el Tráfico Cifrado

MENTO es un nuevo enfoque propuesto en [3], el cual se divide en tres bloques principales: 1) Gestión de memoria, 2) Procedimiento de entrenamiento, y 3) Rectificación del modelo.

El objetivo de la investigación es permitir que los modelos de clasificación puedan aprender nuevas aplicaciones sin olvidar las que ya han aprendido. Esto permite que MEMENTO realice una clasificación de tráfico cifrado en entornos de red en constante evolución.

Su solución fue la división de biflujos, lo que permite que las aplicaciones antiguas se almacenen en una memoria limitada y puedan ser utilizadas en el futuro cuando el modelo necesite reaprender. Un biflujo representa una comunicación bidireccional entre dos extremos de una conexión de red. A diferencia de un flujo unidireccional, el biflujo incluye tanto los paquetes enviados como los recibidos, capturando así el contexto completo de la comunicación. Esta representación resulta útil en tareas de clasificación de tráfico, ya que proporciona una visión más completa del comportamiento de una aplicación en la red.

Entre los distintos métodos utilizados, el que más destacó fue el procedimiento conocido como *Herding*, el cual selecciona los biflujos más cercanos al centroide de cada clase latente. Este método resultó ser más efectivo al equilibrar el rendimiento y el tiempo de ejecución. Sin embargo, el uso de *Herding* no fue suficiente, por lo que se implementó el método de *Jittering*, el cual añade ruido gaussiano a los tiempos de llegada entre cada paquete. En otras palabras, agrega un valor constante a todos los paquetes que llegan en un biflujo. Esto se realizó con el propósito de equilibrar el aprendizaje de las aplicaciones antiguas con las nuevas.

Este enfoque fue probado con los conjuntos de datos *MIRAGE19*, que es un conjunto de datos de tráfico cifrado de aplicaciones móviles para Android, y *CESNET-TLS22*, que es un conjunto de datos genéricos.

Los resultados demostraron que MEMENTO mejoró el rendimiento de la clasificación de tráfico cifrado en aplicaciones móviles en un 16,3 % y en aplicaciones genéricas en un 2,5 %.

9. Metodología

Este trabajo se divide en tres etapas principales: la recreación del modelo base, la implementación de una extensión al modelo, y la comparación de los resultados obtenidos. A continuación, se describen cada una de estas etapas en detalle:

1. Recreación del código base

Como primer paso, se recreará el código presentado en [2], que describe la implementación de una red neuronal multicapa utilizando el algoritmo de entrenamiento *backpropagation* en el lenguaje de programación Python. Este código servirá como base para las siguientes etapas.

Además, se empleará el conjunto de datos *Optical Digits*, el cual será sometido a un proceso de preprocesamiento para eliminar registros inválidos y garantizar la calidad de los datos utilizados en el entrenamiento y validación del modelo.

2. Extensión del modelo base

Posteriormente, se desarrollará una extensión al modelo base, con el objetivo de experimentar con configuraciones más complejas de la red neuronal. Específicamente, se agregarán más de dos capas de pesos duplicados, lo cual se espera que mejore la retención de información al emplear el aprendizaje incremental.

Para este propósito, se realizarán experimentos incrementando gradualmente el número de capas de pesos duplicados, con el fin de analizar el impacto de esta configuración en la capacidad de aprendizaje de la red.

3. Comparación y análisis de resultados

Una vez obtenidos los resultados de las simulaciones, se realizará una comparación entre el rendimiento del modelo base y el modelo extendido. Este análisis incluirá métricas clave, como la precisión del modelo, la tasa de olvido de información y el desempeño en el aprendizaje incremental.

Los resultados permitirán evaluar si las modificaciones introducidas en la arquitectura de la red neuronal mejoran significativamente su desempeño en comparación con la implementación original.

10. Organización del Capitulo

En el capítulo 2 se explicará lo que son y como funcionan las redes neuronales artificiales, así como la función de activación que es un método que utilizan estas. Se describirán los tipos de redes neuronales, tanto de perceptron multicapa como convolucionales, y el algoritmo *backpropagation*. Se mencionará el conjunto de datos *Optdigit* que se utilizará para el entrenamiento y prueba de los algoritmos. Y para terminar, se describirá el aprendizaje incremental y su algoritmo.

En el capítulo 3 se implementará el algoritmo propuesto por [2]. Posteriormente, se verificará su funcionamiento y se analizarán los resultados obtenidos al utilizar los mismos datos que se mencionan en el artículo, con el objetivo de confirmar que el comportamiento del algoritmo coincide con lo descrito por el autor.

En el capítulo 4 se explicará como se se extendió el algoritmo base permitiendo el uso de más de dos capas de pesos duplicados y se aplicaran los mismos datos de entrenamiento y de prueba que al algoritmo base.

Posteriormente, en el capítulo 5 con los resultados obtenidos, se mostrará una comparación de los resultados de ambos trabajos para notar si hubo una reducción significativa en las tasas de aprendizaje. En el capítulo 6 se verán las conclusiones y trabajo futuro.

11. Diagrama de Gantt

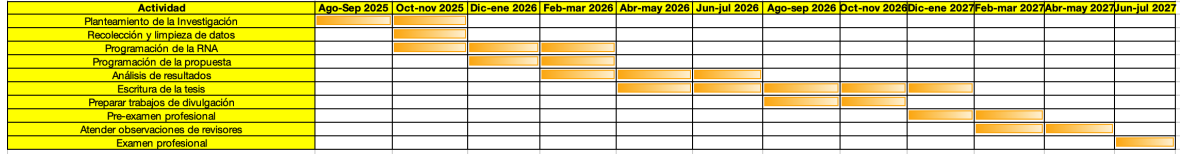


Figura 12: Diagrama de Gantt

A. Fases de la Retropropagación

A.1. Propagación hacia adelante

En la siguiente fase los valores de entrada se introducen a la red, se hace el calculo de la salida y no existe el ajuste de pesos, ya que solo se hace el calculo del valor de activación.

Esto se logra gracias a la multiplicación matricial, donde cada neurona recibe la salida de las neuronas de la capa anterior multiplicadas por sus respectivos pesos, por medio de la siguiente ecuación:

$$z_j = \sum_i (w_{ji} \cdot x_i) + b_j$$

Donde:

- w_{ji} = peso de la conexión entre la neurona i de la capa anterior y la neurona j de la capa actual.
- x_i = salida de la neurona i de la capa anterior.
- b_j = sesgo (bias) de la neurona j .

A.2. Propagación hacia atrás

En este caso se ajusta los pesos de la red propagando el error desde la salida hacia atrás (capas ocultas), usando el gradiente descendente. Para conseguirlo se debe de calcular el error, se debe calcular la perdida de la siguiente manera: $L = \frac{1}{2}(y - \hat{y})^2$. Ya con estos datos se calcula la propagación hacia atrás, usando la regla de la cadena del cálculo diferencial. Aquí se divide en dos, el calculo de la capa de salida y de las capas ocultas.

- Capa de Salida j :

$$\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial z_j}$$

Donde:

- $\frac{\partial L}{\partial \hat{y}_j} = (\hat{y}_j - y_j)$ (derivada de la pérdida).
- $\frac{\partial \hat{y}_j}{\partial z_j}$ depende de la función de activación. Por ejemplo, si es sigmoide:

$$\sigma'(z_j) = \sigma(z_j)(1 - \sigma(z_j))$$

- Capas ocultas:

$$\frac{\partial L}{\partial z_j} = \sum_k \left(\frac{\partial L}{\partial z_k} \cdot w_{kj} \cdot \sigma'(z_j) \right)$$

Aquí, w_{kj} son los pesos de la capa siguiente.

Con los pesos (w_{ji}) y biases (b_j) se puede actualizar usando el gradiente y una tasa de aprendizaje (η):

$$w_{ji} \leftarrow w_{ji} - \eta \cdot \frac{\partial L}{\partial w_{ji}}$$

$$b_j \leftarrow b_j - \eta \cdot \frac{\partial L}{\partial b_j}$$

Donde:

$$\frac{\partial L}{\partial w_{ji}} = \frac{\partial L}{\partial z_j} \cdot x_i \quad \text{y} \quad \frac{\partial L}{\partial b_j} = \frac{\partial L}{\partial z_j}$$

Referencias

- [1] RR Ade and PR Deshmukh. Methods for incremental learning: a survey. *International Journal of Data Mining & Knowledge Management Process*, 3(4):119, 2013.
- [2] John A Bullinaria. Evolved dual weight neural architectures to facilitate incremental learning. In *IJCCI*, pages 427–434, 2009.
- [3] Francesco Cerasuolo, Alfredo Nascita, Giampaolo Bovenzi, Giuseppe Aceto, Domenico Ciunzio, Antonio Pescapè, and Dario Rossi. Memento: A novel approach for class incremental learning of encrypted traffic. *Computer Networks*, 245:110374, 2024.
- [4] Ryan Elwell and Robi Polikar. Incremental learning of concept drift in nonstationary environments. *IEEE Transactions on Neural Networks*, 22(10):1517–1531, 2011.
- [5] Douglas H Fisher. Knowledge acquisition via incremental conceptual clustering. *Machine learning*, 2(2):139–172, 1987.
- [6] Christophe G. Giraud-Carrier. A note on the utility of incremental learning. *AI Commun.*, 13:215–224, 2000.
- [7] H Vega Huerta, A Cortez Vásquez, Ana María Huayna, Luis Alarcón Loayza, and P Romero Naupari. Reconocimiento de patrones mediante redes neuronales artificiales. *Revista de investigación de Sistemas e Informática*, 6(2):17–26, 2009.
- [8] Rodrigo Jacznik, Mariano Tassara, Iván D’Uva, Guillermo Baldino, Roxana Silvia Giandini, and Leopoldo Nahuel. Integrando modelo de aprendizaje supervisado al análisis del desempeño de alumnos en cursos virtuales sobre plataformas moodle. In *XXII Congreso Argentino de Ciencias de la Computación (CACIC 2016)*, 2016.
- [9] Ai-Jun Li. An improved algorithm for incremental learning learn++. In *2008 International Conference on Wavelet Analysis and Pattern Recognition*, volume 1, pages 310–315. IEEE, 2008.
- [10] Vittorio Lippi, Cristian Camardella, Alessandro Filippeschi, and Francesco Porcini. Identification of gait phases with neural networks for smooth transparent control of a lower limb exoskeleton, 2021.
- [11] Yuan Liu. Yuan liu incremental learning in deep neural networks. 2015.
- [12] Darwin Mercado Polo, Luis Pedraza Caballero, and Edinson Martínez Gómez. Comparación de redes neuronales aplicadas a la predicción de series de tiempo. *Prospectiva*, 13(2):88–95, 2015.
- [13] Luis Miralles Pechuán, Dafne A Rosso-Pelayo, and Jorge Brieva. Reconocimiento de dígitos escritos a mano mediante métodos de tratamiento de imagen y modelos de clasificación. *Res. Comput. Sci.*, 93:83–94, 2015.
- [14] V Renganathan. Overview of artificial neural network models in the biomedical domain. *Bratislava Medical Journal/Bratislavské Lekárske Listy*, 120(7), 2019.
- [15] Raúl Rojas. *The Backpropagation Algorithm*, pages 149–182. Springer Berlin Heidelberg, Berlin, Heidelberg, 1996.
- [16] Paloma Royo. Qué son las redes neuronales y cuál es su aplicación en el marketing, 2021.

- [17] Luis A Cruz Salazar, David J Muñoz Aldana, and Juan A Contreras Montes. Implementación de redes neuronales y lógica difusa para la clasificación de patrones obtenidos por un sónar. In *2013 II International Congress of Engineering Mechatronics and Automation (CIIMA)*, pages 1–6. IEEE, 2013.
- [18] Xing Wei, Shaofan Liu, Yaoci Xiang, Zhangling Duan, Chong Zhao, and Yang Lu. Incremental learning based multi-domain adaptation for object detection. *Knowledge-Based Systems*, 210:106420, 2020.
- [19] Beibei Zhao, Tairan Wu, Fang Fang, Lin Wang, Wenzhang Ren, Xu Yang, Zhangjing Ruan, and Xuejin Kou. Prediction method of 5g high-load cellular based on bp neural network. In *2022 8th International Conference on Mechatronics and Robotics Engineering (ICMRE)*, pages 148–151. IEEE, 2022.